

Queue

Queue :- A Queue is a linear structure which follows a particular order in which the operations are performed. The order is FIFO. (First in first out).

A good example of a queue is any queue of Consumers for a resource where the consumers that came first Served first.

The difference bet stack and queue is in removing. In a stack we remove the item the most recently added. in a queue we remove the the item the least recently added.

Application of Queue Data Structure.

Queue is used when things don't have to be processed immediately but have to be processed in First in First out order like Breadth First Search. This property of queue makes it also useful in following kind of scenarios.

- 1) When resource is shared among multiple consumers. e.g. CPU Scheduling, Disk Scheduling.
- 2) When data is transferred Asynchronously (data not necessarily received at same rate as sent) betⁿ two processes. e.g. io Buffer, pipes, File etc.

In operating system :-

- a) Semaphores.
- b) FCFS
- c) Spooling in printer
- d) Buffer for device like keyboard

Queue

Simple queue :-

```
public class SimpleQueue {
```

```
    private int queue[];  
    private int capacity;  
    private int front;  
    private int rear;
```

```
    public SimpleQueue (int capacity)
```

```
    {  
        this.capacity = capacity;  
        this.queue = new int[capacity];  
        this.front = 0;  
        this.rear = -1;  
    }
```

```
    public void enqueue (int data)
```

```
    {  
        if (rear == capacity - 1)
```

```
        {  
            System.out.println ("Queue is full");  
            return -1;  
        }
```

```
    }
```

```
        rear ++;
```

```
        queue[rear] = data;
```

```
    }
```

```
    public int dequeue (int data) {
```

```
        if (front > rear) {
```

```
{ System.out.println("queue is empty");
```

```
int data = queue[front];
```

```
front++;
```

```
}
```

```
public void display () {
```

```
{ if (front > rear)
```

```
{ System.out.println("queue empty");  
return ;
```

```
for (int i = front; i <= rear; i++)
```

```
{ System.out.print (queue[i] + " ");
```

```
}
```

```
public static void main (String args[])
```

```
{ SimpleQueue queue = new SimpleQueue(5)
```

```
queue.enqueue (5);
```

```
queue.enqueue (3);
```

```
queue.enqueue (1);
```

```
queue.display ();
```

```
}  
queue.enqueue (4);  
queue.display ();  
}
```

// Circular queue :-

```
public class circularqueue {
```

```
private int queue[];
```

```
private int front;
```

```
private int rear;
```

```
private int capacity;
```

```
private int size;
```

```
public Circularqueue (int capacity) {
```

```
this.capacity = capacity;
```

```
this.queue = new int [capacity];
```

```
this.front = 0;
```

```
this.rear = 0;
```

```
this.size = 0;
```

```
}
```

```
public void enqueue (int data) {
```

```
{ if (size == capacity)
```

```
{ System.out.println("queue is full");
```

```
return;
```

```
}
```

```

queue [rear] = data;
rear = (rear + 1) % capacity;
size++;
}

```

```

public void dequeue () {

```

```

    if (size == 0) {
        system.out.println("queue is empty");
        return;
    }

```

```

    int data = queue[front];
    front = (front + 1) % capacity;
    size--;

```

```

    return data;
}

```

```

public int size () {
    return this.size;
}

```

```

public void display () {

```

```

    if (size == 0)

```

```

        system.out.println("queue is empty");
        return;
    }

```

```

    for (int i = front; i < size; i++)

```

```

        int data = queue[(front + 1) % capacity];

```

```

    }
    system.out.println(data + " ");
}
system.out.println();
}

```

```

public static void main (String args[]) {

```

```

    CircularQueue queue = new CircularQueue(5);

```

```

    queue.enqueue(5);

```

```

    queue.enqueue(7);

```

```

    queue.enqueue(9);

```

```

    queue.enqueue(10);

```

```

    queue.enqueue(12);

```

```

    queue.display();

```

```

    queue.dequeue();

```

```

    queue.dequeue();

```

```

    queue.display();

```

```

    queue.enqueue(3);

```

```

    queue.enqueue(4);

```

```

    queue.display();

```

```

}
}

```